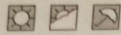


day02

站点	地址	指令	你要看到的
0 序幕		<code>push rbp</code> → <code>mov rbp, rsp</code> → <code>sub rsp, 40</code>	函数开场三件套： RSP 减 0x40 （腾出 64 字节放局部变量），RBP 记下基准点
1 寻址		<code>mov [rbp-8], 5A</code>	栈窗口 <code>rbp-8</code> 处出现 <code>5A 00 00 00</code> (a=90 写内存)
		<code>mov [rbp-C], F</code>	<code>rbp-C</code> 处出现 <code>0F 00 00 00</code> (b=15)
		<code>mov eax, [rbp-C]</code>	RAX 变 0xF （读内存成功）
		<code>add [rbp-8], eax</code>	内存那格 <code>5A</code> → <code>69</code> (90+15=105，直接在内存里加)
2 push/pop		<code>mov rax, 11223344</code> → <code>push rax</code>	RSP 减 8 ，栈顶出现 <code>44 33 22 11</code> (小端!)
		<code>pop rcx</code>	RCX = 0x11223344 ，RSP 加 8 恢复
3 call/ret		<code>call 140001490</code>	按 F7 进函数! 看返回地址 <code>14D9</code> 被压栈，跳进 add5；里面 <code>add eax, 5</code> 后 RAX=110 ； <code>ret</code> 把 <code>14D9</code> 弹回，回到 main
		<code>mov [rbp-10], eax</code>	返回值 110 存进 c 的格子
4 jmp 系列		<code>cmp eax, [rbp-C]</code>	105 和 15 比较， 标志位被更新 （右侧 RFLAGS 变化）

		jle 14E7	看标志: 105>15, 不跳 , 往下走
		mov [rbp-4],64 → jmp 14EE	d=100; jmp 无条件 跳过 else 那格 (14EE 的 C8=200 这次没执行)
终点		printf 参数准备	Windows x64 调用约定 : 格式串→ecx, a→edx, b→r8d, c→r9d, 第 5 个参数 d 压到栈 [rsp+20]
		call printf	控制台输出 a=105 b=1 5 c=110 d=100
收尾		mov eax,0 → add rsp,40	返回值 0 放 eax; 栈空间收回 (RSP 加回 0x40)



Mo Tu We Th Fr Sa Su

Memo No.
Date / /

AccessData Flk Imager

'[]' 去内存取

寄存器里存的是值

mov eax, ebx

~~把 ebx 的值复制到 eax~~

ebx = 0x1000

mov eax, ebx → eax 拿到 0x1000

(地址值)

mov eax, [ebx]

取地址

地址 0x1000 里存着 0x5A

mov eax, [ebx] → eax

拿到 0x5A (值)

PTR

宽度修饰符

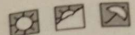
定义宽度大小

mov eax, [rbp-8]

基址 + 偏移

rbp 是基准点

mov / push / pop / call / ret / jmp



Mo Tu We Th

RSP

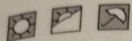
栈顶指

针随着

push

call





Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

RSP

栈顶指针

会随着push/pop移动

push/pop

mov rax 11223344

push rax 先RSP减8,再把值

pop rcx 把[RSP]的值读进RCX, 再RSP+8

call/ret

call 14000140

mov [rbp-10] eax

call X = push 返回地址 + jmp X (压栈+跳走)

ret = pop 返回地址 → RSP (弹栈+跳回)

cmp eax, [rbp-c] 比较 更新标志位

jle 14E7 看标志

jmp 14E1 else

